

Cours 4 ECABD

Scalabilité horizontale et verticale

- **Verticale** : A un instant t j'ai une unité de calcul, et à un instant $t+1$, j'ai une unité de calcul plus puissante. On augmente la capacité de calcul d'une entité
- **Horizontale** : A un instant t une entité de calcul, et à un instant $t+1$, plus qu'une entité de calcul. On multiplie le nombre d'entité de calcul.
 - Moins cher que la scalabilité Verticale

Utilisation d'autres types d'outils :

- **Big Data** -> Solution pour traiter gros volumes de données qui structurées différemment ;
- **NOSQL** -> NotOnlySQL
 - Plusieurs familles différentes :
 - Théorème CAP (Brewer) -> Modélise les contraintes que doivent un respecter un système NOSQL, voici les étapes du plus importants au moins importants :
 - **A** : Availability -> Pouvoir accéder aux données n'importe quand
 - **P** : Partition Tolerance -> Peut fonctionner même si un nœud de notre graphe ne fonctionne plus
 - **C** : Consistency -> Toutes les données doivent être équivalentes et justes sur l'ensemble des nœuds.
 - Types de bases de données en NOSQL
 - Orienté clé-valeur
 - On prend la valeur stockée (entier, char, xml...)
 - Cohérence : On a
 - n = facteur de réplication
 - R = Quorum Lecture
 - W = Quorum Ecriture, nombre maximum d'itération d'écriture avant que l'on puisse considérer que nos données sont à jour (SELECT)
 - Consistance forte -> $R+W>N$

4 Relation entre R , W et n

- Pour garantir une cohérence forte :

$$R + W > n$$

- Cela signifie que les lectures et écritures se chevauchent sur au moins une réplique commune, ce qui assure que les données lues sont à jour.

Exemple :

- $n = 3, R = 2, W = 2 \rightarrow$ cohérence forte car $R + W = 4 > n$.
- $n = 3, R = 1, W = 1 \rightarrow$ cohérence faible, mais meilleure disponibilité.

➔ Consistance peut ralentir les process d'écriture et de lecture.

1 n = facteur de réplication

- Définition : nombre de **répliques d'une donnée** stockées sur différents nœuds du cluster.
- Exemple : $n = 3 \rightarrow$ chaque donnée est copiée sur 3 serveurs différents pour **tolérance aux pannes**.

✓ Plus n est grand $\rightarrow$ meilleure disponibilité, mais plus de stockage utilisé.

2 R = Quorum lecture

- Définition : nombre de **répliques à lire** pour qu'une lecture soit considérée comme valide.
- Permet de **garantir la cohérence** tout en tolérant certaines pannes.

Exemple :

- $n = 3, R = 2 \rightarrow$ pour lire une donnée, le système lit **2 répliques** et retourne la valeur majoritaire.

3 W = Quorum écriture

- Définition : nombre de **répliques qui doivent confirmer l'écriture** pour que celle-ci soit considérée comme réussie.
- Assure que suffisamment de nœuds ont reçu la mise à jour pour la cohérence.

Exemple :

- $n = 3, W = 2 \rightarrow$ l'écriture est validée seulement quand **2 des 3 répliques** ont été mises à jour.

- *Orienté document (Utilisation documents JSON stockage)*
- *Orienté colonnes*
 - *Colonnes statiques*
 - *Colonnes Dynamiques*
 - *Quand une valeur était absente dans un système, on mettait NULL ; Ici sous CASSANDRA la colonne en question n'existe pas.*
 - *Super colonnes -> Groupement de colonnes*
- *Orienté graphe (Système de contrôle Néo4)*
 - *Utilisé par LinkedIn avec le système de relations*
- *3V : Données qui doivent être typées en Volume, en Vitesse, et Variété (le format de typologie de contenu est différent)*
 - *4^{ème} V -> Véracité de l'information -> Il faut que les données apportent de l'intérêt et de la valeur et qu'elles soient véridiques*

Apache Cassandra est une base de données NoSQL distribuée et orientée colonnes.

Trouver des domaines métiers dans lesquels il y'a une correspondance avec chaque Orientation.

<i>Orientation</i>	<i>Exemple de domaine métier</i>	<i>Explication</i>
<i>Clé-Valeur</i>	<i>E-commerce</i>	<i>Client = Clé & Valeur = Panier</i>
<i>Document</i>	<i>Catalogues produits en E-Commerce</i>	<i>Evite à ce que chaque produit suive une seule et même structure</i>
<i>Colonnes</i>	<i>Catalogues Produits E-Commerce</i>	<i>Lecture rapide par colonnes</i>
<i>Graphes</i>	<i>Réseaux Sociaux</i>	<i>Système relationnel de graphes</i>

Orientation	Modèle de stockage	Exemple de SGBD	Cas d'usage typique	Avantages	Limites
Clé-Valeur	Paires clé → valeur	Redis, DynamoDB, Riak	Sessions utilisateurs, paniers e-commerce, cache	Rapidité extrême, simplicité, scalable	Pas de structure complexe, pas de requêtes avancées
Document	Documents (JSON, BSON, XML)	MongoDB, CouchDB, RavenDB	Catalogue produit, profils utilisateurs, contenu web	Flexible (schéma libre), hiérarchique, requêtes riches	Moins efficace pour relations complexes, documents trop gros = lenteur
Colonnes	Données organisées par colonnes (wide-column)	Cassandra, HBase	Big Data, transactions bancaires, logs, catalogues massifs	Optimisé pour requêtes analytiques, très scalable	Modélisation plus complexe, moins flexible que documents
Graphes	Nœuds (entités) + Arêtes (relations)	Neo4j, OrientDB, ArangoDB	Réseaux sociaux, recommandations, fraude bancaire	Représentation naturelle des relations, requêtes relationnelles rapides	Peu adapté aux données massives tabulaires, plus lent sur simples lectures

- **Clé-Valeur** → rapidité, données simples.
- **Document** → flexibilité, données semi-structurées.
- **Colonnes** → performance sur gros volumes et analyses.
- **Graphes** → relations complexes et interconnexions.